

```

# CHAPTER 6 Building Empirical Models

# Refer to § 1-2.3, 1-3, and 2-6.

# § 6-1 Introduction to Empirical Models

# --- 1. Setup ---
if(!require(pacman)) install.packages("pacman")
pacman::p_load(broom, car, ggfortify, ggplot2, goftest, HH, lmtest, mgcv,
  nortest, olsrr, patchwork, rgl)

# Table 6-1 Salt Concentration in Surface Streams and Roadway Area

# --- 2. Input the actual dataset from Table 6-1 ---
roadway_area      <- c(0.19, 0.15, 0.57, 0.40, 0.70, 0.67, 0.63, 0.47,
  0.75, 0.60, 0.78, 0.81, 0.78, 0.69, 1.30, 1.05, 1.52, 1.06, 1.74, 1.62)

salt_concentration <- c( 3.8,  5.9, 14.1, 10.4, 14.6, 14.5, 15.1, 11.9,
  15.5,  9.3, 15.6, 20.8, 14.6, 16.6, 25.6, 20.9, 29.9, 19.6, 31.3, 32.7)

(Watershed_df <- data.frame(
  roadway_area      = roadway_area,
  salt_concentration = salt_concentration
))

summary(Watershed_df)
spm(Watershed_df)

# --- 3. Fit the simple linear regression model ---
regression_model <- lm(salt_concentration ~ roadway_area)

# Figure 6-1 Scatter diagram of the salt concentration in surface streams
# and roadway area data in Table 6-1.

sp(roadway_area, salt_concentration, "xy", xlab = "Roadway area (%)",
  ylab = "Salt conc. (mg/l)")

```

```
# Figure 6-2 The distribution of Y for a given value of x for the salt
# concentration-roadway area data.
```

```
# Figure 6-3 (a) The regression plane for the model  $E(Y|x_1, x_2) = 50 +$ 
#  $10*x_1 + 7*x_2$ . (b) The contour plot.
```

```
f1 <- function(x1, x2) 50 + 10*x1 + 7*x2
```

```
f1.3d <- plot3d(f1, xlab = "x1", ylab = "x2", zlab = "E(Y|x1, x2)",
  col = colorRampPalette(c("#00007F", "blue", "#007FFF", "cyan", "#7FFF7F",
    "yellow", "#FF7F00", "red", "#7F0000")), xlim = c(0, 10),
  ylim = c(0, 10))
```

```
(lv1 <- f1(0, 0) + (0:10)*(f1(10, 10) - f1(0, 0))/10)
```

```
contourLines3d(f1.3d, "z", 11, lv1)
```

```
# Figure 6-4 (a) Three-dimensional plot of regression model  $E(Y|x_1, x_2) =$ 
#  $50 + 10*x_1 + 7*x_2 + 5*x_1*x_2$ . (b) The contour plot.
```

```
f2 <- function(x1, x2) 50 + 10*x1 + 7*x2 + 5*x1*x2
```

```
f2.3d <- plot3d(f2, xlab = "x1", ylab = "x2", zlab = "E(Y|x1, x2)",
  col = colorRampPalette(c("#00007F", "blue", "#007FFF", "cyan", "#7FFF7F",
    "yellow", "#FF7F00", "red", "#7F0000")), xlim = c(0, 10),
  ylim = c(0, 10))
```

```
(lv2 <- f2(0, 0) + (0:10)*(f2(10, 10) - f2(0, 0))/10)
```

```
contourLines3d(f2.3d, "z", 11, lv2, 10000)
```

```
# Figure 6-5 (a) Three-dimensional plot of the regression model  $E(Y|x_1, x_2)$ 
#  $= 800 + 10*x_1 + 7*x_2 - 8.5*x_1^2 - 5*x_2^2 + 4*x_1*x_2$ . (b) The contour plot.
```

```
f3 <- function(x1, x2) 800 + 10*x1 + 7*x2 - 8.5*x1^2 - 5*x2^2 + 4*x1*x2
```

```
f3.3d <- plot3d(f3, xlab = "x1", ylab = "x2", zlab = "E(Y|x1, x2)",  
  col = colorRampPalette(c("#00007F", "blue", "#007FFF", "cyan", "#7FFF7F",  
    "yellow", "#FF7F00", "red", "#7F0000")), xlim = c(0, 10),  
  ylim = c(0, 10))
```

```
(lv3 <- f3(10, 10) + (0:10)*(f3(0, 0) - f3(10, 10))/10)
```

```
contourLines3d(f3.3d, "z", 11, lv3, 10000)
```

```
# § 6-2 Simple Linear Regression
```

```
# Refer to § 6-1.
```

```
# --- 1. Setup ---
```

```
if(!require(pacman)) install.packages("pacman")
pacman::p_load(broom, car, ggfortify, ggplot2, goftest, HH, lmtest, mgcv,
  nortest, olsrr, patchwork, rgl)
```

```
# --- 2. Input the actual dataset from Table 6-1 ---
```

```
roadway_area      <- c(0.19, 0.15, 0.57, 0.40, 0.70, 0.67, 0.63, 0.47,
  0.75, 0.60, 0.78, 0.81, 0.78, 0.69, 1.30, 1.05, 1.52, 1.06, 1.74, 1.62)
```

```
salt_concentration <- c( 3.8,  5.9, 14.1, 10.4, 14.6, 14.5, 15.1, 11.9,
  15.5,  9.3, 15.6, 20.8, 14.6, 16.6, 25.6, 20.9, 29.9, 19.6, 31.3, 32.7)
```

```
(Watershed_df <- data.frame(
  roadway_area      = roadway_area,
  salt_concentration = salt_concentration
))
```

```
summary(Watershed_df)
spm(Watershed_df)
```

```
# --- 3. Fit the simple linear regression model ---
```

```
regression_model <- lm(salt_concentration ~ roadway_area)
```

```
# Figure 6-1 Scatter diagram of the salt concentration in surface streams
# and roadway area data in Table 6-1.
```

```
sp(roadway_area, salt_concentration, "xy", xlab = "Roadway area (%)",
  ylab = "Salt conc. (mg/l)")
```

```
# Figure 6-2 The distribution of Y for a given value of x for the salt
# concentration-roadway area data.
```

```
# § 6-2.1 Least Squares Estimation
```

```
# Figure 6-6 Deviations of the data from the estimated regression model.
```

```
regresidplot(roadway_area, salt_concentration, xlab = "Roadway area (%)",  
  ylab = "Salt Conc. (mg/l)")
```

```
regresidplot(roadway_area, salt_concentration, T,  
  xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
```

```
regresidplot(roadway_area, salt_concentration, "square",  
  xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
```

```
# EXAMPLE 6-1 Salt Concentration and Roadway Data
```

```
# Refer to Table 6-2.
```

```
# Figure 6-7 Scatter diagram of salt concentration y versus roadway area x  
# and the fitted regression model.
```

```
# ols_regress(salt_concentration ~ roadway_area, Watershed)  
# (lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))  
# summary(lm.Watershed)  
# coef(lm.Watershed)  
# (newdata <- data.frame(roadway_area = 1.25))  
# predict(lm.Watershed, newdata, T, NULL, Inf, "c")  
# predict(lm.Watershed, newdata, T, NULL, Inf, "p")
```

```
# Table 6-2 Minitab Regression Analysis Output for Salt Concentration and  
# Roadway Data
```

```
ols_regress(salt_concentration ~ roadway_area, Watershed)
```

```
(lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))
```

```
summary(lm.Watershed)
```

```

coef(lm.Watershed)

anova(lm.Watershed)

lht(lm.Watershed, "roadway_area")

fitted(lm.Watershed)

predict(lm.Watershed, Watershed, T)

resid(lm.Watershed)

confint(lm.Watershed)

(newdata <- data.frame(roadway_area = 1.25))

predict(lm.Watershed, newdata, T, NULL, Inf, "c")

predict(lm.Watershed, newdata, T, NULL, Inf, "p")

ci.plot(lm.Watershed, main = "95% confidence and prediction intervals",
        xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")

# EXAMPLE 6-1 (continued)

# Refer to Table 6-2.

# ols_regress(salt_concentration ~ roadway_area, Watershed)
# (lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))
# summary(lm.Watershed)
# anova(lm.Watershed)
# lht(lm.Watershed, "roadway_area")

# EXAMPLE 6-1 (continued)

# Refer to Table 6-2.

```

```

# ols_regress(salt_concentration ~ roadway_area, Watershed)
# (lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))
# summary(lm.Watershed)

# --- A. R-Squared and Adjusted R-Squared ---
# 1. Fit the ordinary least-squares simple linear regression model
regression_model <- lm(salt_concentration ~ roadway_area)
model_summary <- summary(regression_model)

# 2. Extract the R-squared and adjusted R-squared parameters
r_squared <- model_summary$r.squared
adj_r_squared <- model_summary$adj.r.squared

# 3. Print out the structured variance explanation report
cat("=====\n")
cat("          MODEL EXPLANATORY POWER VARIANCE SUMMARY          \n")
cat("=====\n")
cat(sprintf("Sample Size (n):                %d\n",
length(salt_concentration)))
cat(sprintf("Coefficient of Determination (R^2):    %s\n",
signif(r_squared, 4)))
cat(sprintf("Adjusted R-Squared (Adj R^2):          %s\n",
signif(adj_r_squared, 4)))
cat("=====\n")

```

§ 6-2.2 Testing Hypotheses in Simple Linear Regression

Refer to § 6-2.1.

Figure 6-8 The hypothesis $H_0: \beta_1 = 0$ is not rejected.

Figure 6-9 The hypothesis $H_0: \beta_1 = 0$ is rejected.

EXAMPLE 6-2 Salt Concentration and Roadway Data

Refer to Table 6-2.

```
# ols_regress(salt_concentration ~ roadway_area, Watershed)
# (lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))
# summary(lm.Watershed)
```

Table 6-3 Analysis of Variance for Testing Significance of Regression

EXAMPLE 6-2 (continued)

Refer to Table 6-2.

```
# ols_regress(salt_concentration ~ roadway_area, Watershed)
# (lm.Watershed <- lm(salt_concentration ~ roadway_area, Watershed))
# summary(lm.Watershed)
```


§ 6-2.3 Confidence Intervals in Simple Linear Regression

Refer to § 6-2.1.

EXAMPLE 6-3 Salt Concentration and Roadway Data

Refer to Table 6-2.

```
# ols_regress(salt_concentration ~ roadway_area, Watershed)
```

```
# confint(lm.Watershed)
```

Confidence Interval on the Mean Response

EXAMPLE 6-4 Salt Concentration and Roadway Data

Refer to Table 6-2.

```
# ci.plot(lm.Watershed, main = "95% confidence and prediction intervals",
```

```
#   xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
```

```
# (newdata <- data.frame(roadway_area = 1.25))
```

```
# predict(lm.Watershed, newdata, T, NULL, Inf, "c")
```

Figure 6-10 Scatter diagram of salt concentration and roadway area from

Example 6-1 with fitted regression line and 95% confidence limits on

$\mu_{Y|x_0}$.

§ 6-2.4 Prediction of a Future Observation

Refer to § 6-2.1 and 6-2.3.

EXAMPLE 6-5 Salt Concentration and Roadway Data

Refer to Table 6-2.

```
# (newdata <- data.frame(roadway_area = 1.25))
```

```
# predict(lm.Watershed, newdata, T, NULL, Inf, "p")
```

```
# ci.plot(lm.Watershed, main = "95% confidence and prediction intervals",
```

```
#   xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")
```

Figure 6-11 Scatter diagram of salt concentration-roadway area data from

Example 6-1 with fitted regression line, 95% prediction limits (outer

lines), and 95% confidence limits on $\mu_{Y|x_0}$.

```
# § 6-2.5 Checking Model Adequacy
```

```
# Refer to § 6-2.1.
```

```
# --- A. Diagnostic Plots ---
```

```
# 1. Configure the plotting region to a 2-row, 2-column matrix grid layout.
```

```
par(mfrow = c(2, 2))
```

```
# 2. Generate the 4 classic diagnostic plots automatically.
```

```
# Plot 1: Residuals vs. Fitted (Checks linearity and constant conditional  
# variance.)
```

```
# Plot 2: Normal Q-Q (Checks conditional normality assumption for errors)
```

```
# Plot 3: Scale-Location (Checks homoscedasticity via the absolute value of  
# standardized residuals)
```

```
# Plot 4: Cook's Distance (Identifies influential points via Cook's D  
# boundaries)
```

```
plot(regression_model,  
      which = 1:4,  
      pch = 19,  
      col = "black",  
      sub.caption = "")
```

```
# 3. Reset the graphics grid parameter back to standard default.
```

```
par(mfrow = c(1, 1))
```

```
# --- B. Diagnostic Plots (ggplot2) ---
```

```
# 1. Use broom::augment() to convert internal model diagnostics into a  
# clean data frame.
```

```
diagnostic_data <- augment(regression_model)
```

```
diagnostic_data$obs_index <- 1:nrow(diagnostic_data) # Add individual  
# observation index numbers.
```

```
# 2. Handcraft the 4 classic diagnostic charts using standard modern
```

```
# syntax. This isolates line thickness and completely bypasses aesthetic  
# warning blocks.
```

```

# Plot 1: Residuals vs. Fitted
p1 <- ggplot(diagnostic_data, aes(x = .fitted, y = .resid)) +
  geom_point(color = "black", size = 2) +
  geom_smooth(method = "loess", formula = y ~ x, color = "red", se = FALSE,
    linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black",
    linewidth = 0.5) +
  labs(title = "Residuals vs. Fitted", x = "Fitted values",
    y = "Residuals")

# Plot 2: Normal Q-Q
p2 <- ggplot(diagnostic_data, aes(sample = .std.resid)) +
  stat_qq(color = "black", size = 2) +
  stat_qq_line(linetype = "dashed", color = "red", linewidth = 0.8) +
  labs(title = "Normal Q-Q", x = "Theoretical Quantiles",
    y = "Standardized Residuals")

# Plot 3: Scale-Location
p3 <- ggplot(diagnostic_data,
  aes(x = .fitted, y = sqrt(abs(.std.resid)))) +
  geom_point(color = "black", size = 2) +
  geom_smooth(method = "loess", formula = y ~ x, color = "red", se = FALSE,
    linewidth = 0.8) +
  labs(title = "Scale-Location", x = "Fitted values",
    y = "sqrt(|Standardized Residuals|)")

# Plot 4: Cook's Distance (Bar plot format for tracking influential
# observations)
p4 <- ggplot(diagnostic_data, aes(x = obs_index, y = .cooks_d)) +
  geom_col(fill = "black", width = 0.5) +
  labs(title = "Cook's Distance", x = "Obs. Number", y = "Cook's distance")

# 3. Assemble the separate quadrants into a single 2 x 2 matrix grid
# utilizing patchwork math.
modern_diagnostic_grid <- (p1 | p2) / (p3 | p4)

```

```

# 4. Render the final composite graphics output window safely without
# warnings
print(modern_diagnostic_grid)

# Figure 6-12 Patterns for residual plots: (a) satisfactory, (b) funnel,
# (c) double bow, (d) nonlinear. Horizontal axis may be time, yihat, or xi.

# EXAMPLE 6-6 Salt Concentration and Roadway Data

# Refer to Table 6-2.

# Figure 6-13 Normal probability plot of residuals from the salt
# concentration-roadway area regression model.

# Figure 6-14 Plot of residuals versus fitted values for the salt
# concentration-roadway area regression model.

# 1. Extract diagnostics: fitted values and ordinary residuals
fitted_values      <- predict(regression_model)
residuals_e        <- resid(regression_model)
observation_order  <- 1:length(residuals_e)

# 2. Set up a 1-row, 2-column plotting layout
par(mfrow = c(1, 2))

# Plot A: Residuals vs. Fitted Values (Checks Constant Conditional Variance
# & Linearity)
plot(fitted_values, residuals_e,
     main = "Residuals vs. Fitted Values",
     xlab = "Fitted Values ( $\hat{y}$ )",
     ylab = "Residuals (e)",
     pch = 19, col = "black")
abline(h = 0, lty = 2, col = "black") # Add baseline reference

```

```

# Plot B: Residuals vs. Observation Order (Checks Independence /
# Autocorrelation)
plot(observation_order, residuals_e,
     main = "Residuals vs. Time/Order",
     xlab = "Observation Order",
     ylab = "Residuals (e)",
     type = "b", pch = 19, col = "black") # "b" connects points with lines
abline(h = 0, lty = 2, col = "black") # Add baseline reference

# Reset plotting layout parameters back to default
par(mfrow = c(1, 1))

# 3. Extract specific diagnostic metrics for summary output
summary_stats <- summary(regression_model)
ms_e <- summary_stats$sigma^2 # Mean Square Error (estimated variance  $\sigma^2$ )

cat("=====\n")
cat("          RESIDUAL VARIANCE DIAGNOSTIC SUMMARY          \n")
cat("=====\n")
cat(sprintf("Sample Size (n):                %d\n",
length(salt_concentration)))
cat(sprintf("Mean Square Error (MS_E or sigma^2): %s\n", signif(ms_e, 4)))
cat(sprintf("Maximum Absolute Residual value:    %s\n",
signif(max(abs(residuals_e)), 4)))
cat("=====\n")

# 4. Generate the Normal Probability Plot
qqnorm(residuals_e,
      main = "Normal Probability Plot of Residuals",
      xlab = "Theoretical Quantiles",
      ylab = "Sample Quantiles (Residuals)",
      pch = 19, col = "black")

# Add a reference line passing through the first and third quartiles
qqline(residuals_e, lty = 2, col = "black")

```

```

# 5. Extract and print descriptive metrics to verify shape properties
# Calculate sample skewness and kurtosis manually for the summary report
n <- length(residuals_e)
mean_e <- mean(residuals_e)
sd_e <- sd(residuals_e)

skewness <- (sum((residuals_e - mean_e)^3) / n) /
  (sum((residuals_e - mean_e)^2) / n)^(3/2)
kurtosis <- (sum((residuals_e - mean_e)^4) / n) /
  (sum((residuals_e - mean_e)^2) / n)^2

cat("=====\n")
cat("          RESIDUAL NORMALITY SHAPE SUMMARY          \n")
cat("=====\n")
cat(sprintf("Sample Size (n):                %d\n", n))
cat(sprintf("Residual Mean (should be zero): %s\n", signif(mean_e, 4)))
cat(sprintf("Residual Sample Skewness:         %s\n", signif(skewness, 4)))
cat(sprintf("Residual Sample Kurtosis:          %s\n", signif(kurtosis, 4)))
cat("=====\n")

# 6. Perform the formal Shapiro-Wilk normality test
shapiro_results <- shapiro.test(residuals_e)

# 7. Print out the structured normality report
cat("=====\n")
cat("          SHAPIRO-WILK NORMALITY TEST REPORT          \n")
cat("=====\n")
cat(sprintf("Sample Size (n):    %d\n", length(residuals_e)))
cat(sprintf("Test Statistic (W): %s\n",
  signif(shapiro_results$statistic, 4)))
cat(sprintf("Exact Test P-Value: %s\n",
  signif(shapiro_results$p.value, 4)))
cat("=====\n")

```

```

# 8. Extract ordinary residuals and compute standardized residuals
# rstandard() accounts for the leverage profile h_ii of each individual
# observation: d_i = e_i / sqrt(MS_E * (1 - h_ii))
standardized_res <- rstandard(regression_model)

# 9. Create a clean matrix layout for the output report
diagnostic_table <- data.frame(
  Observation = 1:length(standardized_res),
  Roadway_Area = roadway_area,
  Salt_Conc = salt_concentration,
  Standardized_Residual = standardized_res
)

# 10. Print the formatted table matching 4 significant figures
cat("=====\n")
cat("          STANDARDIZED RESIDUAL DIAGNOSTIC TABLE          \n")
cat("=====\n")
for(i in 1:nrow(diagnostic_table)) {
  cat(sprintf("Obs %2d | X: %5.2f | Y: %5.1f | Standardized Residual:
%s\n",
              diagnostic_table$Observation[i],
              diagnostic_table$Roadway_Area[i],
              diagnostic_table$Salt_Conc[i],
              signif(diagnostic_table$Standardized_Residual[i], 4)))
}
cat("=====\n")

# 11. Compute the square root of the absolute standardized residuals
sqrt_abs_std_res <- sqrt(abs(standardized_res))

# 12. Generate the Scale-Location plot
plot(fitted_values, sqrt_abs_std_res,
     main = "Scale-Location Plot",
     xlab = "Fitted Values (y^)",
     ylab = "Square Root of |Standardized Residuals|",
     pch = 19, col = "black")

```



```

# Add a smooth trend line (lowess) to reveal structural variance patterns
lines(lowess(fitted_values, sqrt_abs_std_res), lty = 1, lwd = 2,
      col = "black")

# 13. Calculate Cook's Distance for each individual observation
cooks_d <- cooks.distance(regression_model)

# 14. Establish standard threshold rules for influential points
# A common screening threshold rule is 4 / n, where n is the sample size
sample_size <- length(salt_concentration)
threshold <- 4 / sample_size

# 15. Print the formatted diagnostics table matching 4 significant figures
cat("=====\n")
cat(" COOK'S DISTANCE DIAGNOSTIC TABLE \n")
cat("=====\n")
cat(sprintf("Sample Size (n): %d | Influential Threshold (4/n): %s\n",
  sample_size, signif(threshold, 4)))
cat("-----\n")

for(i in 1:sample_size) {
  status <- if(cooks_d[i] > threshold) " * INFLUENTIAL *" else "Normal"
  cat(sprintf("Obs %2d | Cook's D: %s \t| Status: %s\n",
    i,
    signif(cooks_d[i], 4),
    status))
}
cat("=====\n")

# Figure 6-15 Effect of an outlier.

# --- C. Formal Partial F-Test ---
# 1. Fit the Reduced Model (First-Order Linear)
model_reduced <- lm(salt_concentration ~ roadway_area)

# 2. Fit the Full Model (Second-Order Polynomial / Quadratic)
model_full <- lm(salt_concentration ~ poly(roadway_area, 2, raw = TRUE))

```

```

# 3. Perform the formal Partial F-Test
partial_f_test <- anova(model_reduced, model_full)

# 4. Extract specific diagnostic metrics for the 4-significant-figures
# report
# The anova() output contains rows for both models. Row 2 captures the
# partial differences.
df_reduced <- partial_f_test$Res.Df[1]
df_full    <- partial_f_test$Res.Df[2]
df_diff    <- partial_f_test$Df[2] # Degrees of freedom for the extra term
# (2 - 1 = 1)

ss_reduced <- partial_f_test$RSS[1]
ss_full    <- partial_f_test$RSS[2]
ss_diff    <- partial_f_test$`Sum of Sq`[2] # Extra Sum of Squares:
# SS_Reduced - SS_Full

f_statistic <- partial_f_test$F[2]
p_value     <- partial_f_test$`Pr(>F)`[2]

# 5. Derive the exact F critical value for comparison
alpha       <- 0.05
f_critical  <- qf(1 - alpha, df_diff, df_full)

```

```

# 6. Print out the structured Partial F-Test Report
cat("=====\\n")
cat("          FORMAL PARTIAL F-TEST REPORT          \\n")
cat("=====\\n")
cat(sprintf("Reduced Model Residual Df:          %d\\n", df_reduced))
cat(sprintf("Full Model Residual Df:          %d\\n", df_full))
cat(sprintf("Reduced Model Residual SS (SS_E):      %s\\n",
  signif(ss_reduced, 4)))
cat(sprintf("Full Model Residual SS (SS_E):          %s\\n",
  signif(ss_full, 4)))
cat(sprintf("Extra Sum of Squares (SS_Drop):          %s\\n",
  signif(ss_diff, 4)))
cat("\\n--- Test Statistic and Rejection Threshold ---\\n")
cat(sprintf("Calculated Partial F-Statistic (f0):    %s\\n",
  signif(f_statistic, 4)))
cat(sprintf("F Rejection Critical Value (f_alpha): %s\\n",
  signif(f_critical, 4)))
cat(sprintf("Exact Test P-Value:                          %.4f\\n", p_value))
cat("=====\\n")

# --- D. Quadratic T-Test and Partial F-Test ---
# 1. Fit the Full Model (Second-Order Polynomial / Quadratic)
model_full <- lm(salt_concentration ~ poly(roadway_area, 2, raw = TRUE))
summary_full <- summary(model_full)

# 2. Extract the t-statistic for the quadratic term (beta2_hat)
# In poly(roadway_area, 2, raw = TRUE), the terms are named 'X' and 'X^2'
quadratic_t0 <-
  summary_full$coefficients["poly(roadway_area, 2, raw = TRUE)2", "t value"]

# 3. Square the extracted t-statistic
squared_t0 <- quadratic_t0^2

```

```

# 4. Re-fit the Reduced Model and extract the Partial F-statistic (f0)
model_reduced <- lm(salt_concentration ~ roadway_area)
partial_f_test <- anova(model_reduced, model_full)
partial_f0 <- partial_f_test$F[2] # Row 2 captures the partial
# difference statistic

# 6. Print out the structured verification report
cat("=====\\n")
cat("          QUADRATIC T-TEST AND PARTIAL F-TEST          \\n")
cat("=====\\n")
cat(sprintf("Quadratic Coefficient t-Test Statistic (t0): %s\\n",
  signif(quadratic_t0, 4)))
cat(sprintf("Square of the Quadratic t-Statistic (t0^2): %s\\n",
  signif(squared_t0, 4)))
cat(sprintf("Calculated Partial F-Test Statistic (f0): %s\\n",
  signif(partial_f0, 4)))
cat("\\n--- Identity Check ---\\n")
cat(sprintf("Is t0^2 exactly equal to f0? %s\\n",
  isTRUE(all.equal(squared_t0, partial_f0))))
cat("=====\\n")

# --- E. Second-Order Polynomial Regression Model ---
# 1. Create a unified data frame for plotting
plot_data_poly <- data.frame(X = roadway_area, Y = salt_concentration)

# 2. Fit the second-order polynomial (quadratic) regression model
# poly(X, 2, raw = TRUE) sets up the terms: X + X^2
quadratic_model <- lm(Y ~ poly(X, 2, raw = TRUE), data = plot_data_poly)
quad_summary <- summary(quadratic_model)

# 3. Generate a dense grid for smooth parabolic curve rendering
dense_x <- seq(min(plot_data_poly$X), max(plot_data_poly$X),
  length.out = 200)
prediction_grid <- data.frame(X = dense_x)

```

```

# Extract 95% Confidence and Prediction Intervals for the quadratic fit
ci_quad <- predict(quadratic_model, newdata = prediction_grid,
  interval = "confidence", level = 0.95)
pi_quad <- predict(quadratic_model, newdata = prediction_grid,
  interval = "prediction", level = 0.95)

# Combine boundaries into a single tracking data frame
interval_bands_quad <- data.frame(
  X      = dense_x,
  Fitted = ci_quad[, "fit"],
  CI_Lwr = ci_quad[, "lwr"],
  CI_Upr = ci_quad[, "upr"],
  PI_Lwr = pi_quad[, "lwr"],
  PI_Upr = pi_quad[, "upr"]
)

# 4. Calculate exact target metrics at x0 = 1.25% for comparison
target_x0 <- data.frame(X = 1.25)
target_pi <- predict(quadratic_model, newdata = target_x0,
  interval = "prediction", level = 0.95)
quad_lwr <- target_pi[1, "lwr"]
quad_upr <- target_pi[1, "upr"]

# 5. Build the layered polynomial ggplot visualization
polynomial_plot <- ggplot() +
  # Layer A: Plot the raw observation scatter points
  geom_point(data = plot_data_poly, aes(x = X, y = Y), color = "black",
    size = 2.5) +

  # Layer B: Add 95% Prediction Intervals (Outer dashed lines)
  geom_line(data = interval_bands_quad, aes(x = X, y = PI_Lwr),
    linetype = "dashed", color = "black", linewidth = 0.8) +
  geom_line(data = interval_bands_quad, aes(x = X, y = PI_Upr),
    linetype = "dashed", color = "black", linewidth = 0.8) +

```

```

# Layer C: Add 95% Confidence Intervals (Inner shaded ribbon)
geom_ribbon(data = interval_bands_quad,
  aes(x = X, ymin = CI_Lwr, ymax = CI_Upr), alpha = 0.15,
  fill = "black") +

# Layer D: Superimpose the smooth quadratic fitted model curve
geom_line(data = interval_bands_quad, aes(x = X, y = Fitted),
  linetype = "solid", color = "black", linewidth = 1.0) +

# Layer E: Example 6-5 comparative reference markers at x0 = 1.25%
geom_vline(xintercept = 1.25, linetype = "dotted", color = "black",
  linewidth = 0.8) +
annotate("point", x = 1.25, y = quad_lwr, color = "black", size = 3,
  shape = 21, fill = "white", stroke = 1.5) +
annotate("point", x = 1.25, y = quad_upr, color = "black", size = 3,
  shape = 21, fill = "white", stroke = 1.5) +
annotate("text", x = 1.29, y = quad_lwr,
  label = paste("Quad Lwr =", signif(quad_lwr, 4)), hjust = 0,
  size = 3.5, fontface = "bold") +
annotate("text", x = 1.29, y = quad_upr,
  label = paste("Quad Upr =", signif(quad_upr, 4)), hjust = 0,
  size = 3.5, fontface = "bold") +

# Layer F: Configure labels and formatting
labs(
  title = "Second-Order Polynomial Regression Model Fit",
  subtitle = "Model:  $Y = 1.047 + 22.84X - 3.012X^2$  | Shaded: 95% CI |
Dashed: 95% PI",
  x = "Roadway Area (%)",
  y = "Salt Concentration (mg/l)"
) +
theme(
  plot.title = element_text(face = "bold", size = 12, hjust = 0.5),
  plot.subtitle = element_text(size = 10, hjust = 0.5, face = "italic"),
  axis.title = element_text(face = "bold", size = 11)
)

```

```

# 6. Render the final composite graphics output window
print(polynomial_plot)

# 7. Extract and print quadratic coefficients details for tracking
# verification
beta0 <- quad_summary$coefficients[1, 1]
beta1 <- quad_summary$coefficients[2, 1]
beta2 <- quad_summary$coefficients[3, 1]
r2_q <- quad_summary$r.squared

cat("--- Quadratic Model Parameter Details ---\n")
cat(sprintf("Intercept (beta0_hat): %s\n", signif(beta0, 4)))
cat(sprintf("Linear Coeff (beta1_hat): %s\n", signif(beta1, 4)))
cat(sprintf("Quad Coeff (beta2_hat): %s\n", signif(beta2, 4)))
cat(sprintf("Model Fit Quality (R^2): %s\n", signif(r2_q, 4)))

# --- F. Diagnostic Plots (Second-Order Polynomial Regression Model) ---
# 1. Fit the Full Model (Second-Order Polynomial / Quadratic)
# poly(..., raw = TRUE) keeps the variables on the original raw scale
model_quadratic <- lm(salt_concentration ~ poly(roadway_area, 2,
raw = TRUE))

# 3. Configure the plotting region to a 1-row, 2-column matrix grid layout
# This isolates the two primary plots needed to evaluate distribution
# improvements
par(mfrow = c(1, 2))

# Plot A: Residuals vs. Fitted Plot (Checks Linearity and Homoscedasticity)
plot(model_quadratic,
      which = 1,
      pch = 19,
      col = "black",
      sub.caption = "")

```

```

# Plot B: Normal Q-Q Plot (Checks Conditional Error Normality)
plot(model_quadratic,
      which = 2,
      pch = 19,
      col = "black",
      sub.caption = "")

# Reset plotting layout parameters back to standard default
par(mfrow = c(1, 1))

# --- G. Lack-of-Fit Test ---
# 1. Fit the Proposed Reduced Model (First-Order Linear Model)
model_linear <- lm(salt_concentration ~ roadway_area)

# 2. Fit the Saturated Model (Groups unique X values as separate factor
# levels)
# This model captures all available replicate information to compute Pure
# Error
model_saturated <- lm(salt_concentration ~ factor(roadway_area))

# 3. Perform the formal Lack-of-Fit ANOVA Decompositions
lof_table <- anova(model_linear, model_saturated)

# 4. Extract components for our clean 4-significant-figures layout matrix
# Pure Error metrics are derived from the saturated model row (Row 2)
df_pure_error <- lof_table$Res.Df[2]
ss_pure_error <- lof_table$RSS[2]
ms_pure_error <- ss_pure_error / df_pure_error

# Lack-of-Fit metrics represent the drop between the two specifications
df_lack_of_fit <- lof_table$Df[2]
ss_lack_of_fit <- lof_table$`Sum of Sq`[2]
ms_lack_of_fit <- ss_lack_of_fit / df_lack_of_fit

# Calculate the formal Lack-of-Fit F-test statistic
f0_lof <- ms_lack_of_fit / ms_pure_error
p_val_lof <- pf(f0_lof, df_lack_of_fit, df_pure_error, lower.tail = FALSE)

```



```

# 5. Extract the alternative Model Comparison metric (Partial F-Test for
# the Quadratic model)
model_quadratic <- lm(salt_concentration ~ poly(roadway_area, 2,
  raw = TRUE))
quad_table      <- anova(model_linear, model_quadratic)
p_val_quad      <- quad_table$`Pr(>F)`[2]

# 6. Print the official Lack-of-Fit Report
cat("=====\\n")
cat("          FORMAL PURE ERROR LACK-OF-FIT REPORT          \\n")
cat("=====\\n")
cat(sprintf("Lack-of-Fit Sum of Squares (SS_LF):      %s\\n",
  signif(ss_lack_of_fit, 4)))
cat(sprintf("Pure Error Sum of Squares (SS_PE):      %s\\n",
  signif(ss_pure_error, 4)))
cat(sprintf("Lack-of-Fit Degrees of Freedom (df_LF): %d\\n",
  df_lack_of_fit))
cat(sprintf("Pure Error Degrees of Freedom (df_PE):  %d\\n",
  df_pure_error))
cat("\\n--- Test Statistic and Rejection Threshold ---\\n")
cat(sprintf("Calculated Lack-of-Fit F-Statistic:      %s\\n",
  signif(f0_lof, 4)))
cat(sprintf("Lack-of-Fit Test P-Value:                  %.4f\\n", p_val_lof))
cat("-----\\n")
cat(sprintf("Alternative Quadratic Term P-Value:          %.4f\\n", p_val_quad))
cat("=====\\n")

# --- H. Log-Transformed Regression Model ---
# 1. Create a data frame and compute the natural log of the response
# variable
plot_data_log <- data.frame(
  X = roadway_area,
  Y = salt_concentration,
  LogY = log(salt_concentration) # ln(Y) transformation
)

```

```

# 2. Fit the log-transformed simple linear regression model
log_model <- lm(LogY ~ X, data = plot_data_log)
log_summary <- summary(log_model)

# 3. Generate a dense grid for smooth curve rendering
dense_x <- seq(min(plot_data_log$X), max(plot_data_log$X),
  length.out = 200)
prediction_grid <- data.frame(X = dense_x)

# Extract 95% Confidence and Prediction Intervals in the log transformed
# scale
ci_log <- predict(log_model, newdata = prediction_grid,
  interval = "confidence", level = 0.95)
pi_log <- predict(log_model, newdata = prediction_grid,
  interval = "prediction", level = 0.95)

# 4. Construct the interval tracking data frame (Both Scales)
interval_bands_log <- data.frame(
  X          = dense_x,
  # Transformed scale metrics: ln(Y)
  Log_Fitted = ci_log[, "fit"],
  Log_CI_Lwr = ci_log[, "lwr"],
  Log_CI_Upr = ci_log[, "upr"],
  Log_PI_Lwr = pi_log[, "lwr"],
  Log_PI_Upr = pi_log[, "upr"],

  # Back-transformed original scale metrics via exponentiation: exp(ln(Y))
  Orig_Fitted = exp(ci_log[, "fit"]),
  Orig_CI_Lwr = exp(ci_log[, "lwr"]),
  Orig_CI_Upr = exp(ci_log[, "upr"]),
  Orig_PI_Lwr = exp(pi_log[, "lwr"]),
  Orig_PI_Upr = exp(pi_log[, "upr"])
)

```

```

# 5. Calculate exact high-precision target metrics at x0 = 1.25% for
# comparison
target_x0    <- data.frame(X = 1.25)
target_pi    <- predict(log_model, newdata = target_x0,
  interval = "prediction", level = 0.95)
orig_pi_lwr <- exp(target_pi[1, "lwr"])
orig_pi_upr = exp(target_pi[1, "upr"])

# 6. Build the visualization mapping back to the original data scale (Fixed
# Version)
transformed_regression_plot <- ggplot() +
  # Layer A: Plot the raw data points on their original scale
  geom_point(data = plot_data_log, aes(x = X, y = Y), color = "black",
    size = 2.5) +

  # Layer B: Back-transformed 95% prediction intervals (creates curved,
  # opening lines)
  geom_line(data = interval_bands_log, aes(x = X, y = Orig_PI_Lwr),
    linetype = "dashed", color = "black", linewidth = 0.8) +
  geom_line(data = interval_bands_log, aes(x = X, y = Orig_PI_Upr),
    linetype = "dashed", color = "black", linewidth = 0.8) +

  # Layer C: Back-transformed shaded 95% confidence intervals
  geom_ribbon(data = interval_bands_log,
    aes(x = X, ymin = Orig_CI_Lwr, ymax = Orig_CI_Upr), alpha = 0.15,
    fill = "black") +

  # Layer D: Back-transformed non-linear fitted regression curve
  geom_line(data = interval_bands_log, aes(x = X, y = Orig_Fitted),
    linetype = "solid", color = "black", linewidth = 1.0) +

```

```

# Layer E: Example 6-5 comparative reference markers at x0 = 1.25%
geom_vline(xintercept = 1.25, linetype = "dotted", color = "black",
  linewidth = 0.8) +
annotate("point", x = 1.25, y = orig_pi_lwr, color = "black", size = 3,
  shape = 21, fill = "white", stroke = 1.5) +
annotate("point", x = 1.25, y = orig_pi_upr, color = "black", size = 3,
  shape = 21, fill = "white", stroke = 1.5) +
annotate("text", x = 1.29, y = orig_pi_lwr,
  label = paste("New Lwr =", signif(orig_pi_lwr, 4)), hjust = 0,
  size = 3.5, fontface = "bold") +
annotate("text", x = 1.29, y = orig_pi_upr,
  label = paste("New Upr =", signif(orig_pi_upr, 4)), hjust = 0,
  size = 3.5, fontface = "bold") +

# Layer F: Labels and formatting
labs(
  title = "Log-Transformed Regression Model Back-Transformed to Original
Scale",
  subtitle = "Model Equation:  $\ln(Y) = 1.638 + 1.157X$  | Shaded: 95% CI |
Dashed: 95% PI",
  x = "Roadway Area (%)",
  y = "Salt Concentration (mg/l)"
) +
theme(
  plot.title = element_text(face = "bold", size = 11, hjust = 0.5),
  plot.subtitle = element_text(size = 9, hjust = 0.5, face = "italic"),
  axis.title = element_text(face = "bold", size = 11)
)

# 7. Render the final plot window safely without object errors
print(transformed_regression_plot)

```

```

# 8. Print transformed summary parameter details for logging
cat("--- Log-Transformed Regression Summary Parameter Details ---\n")
cat(sprintf("Intercept (beta0_hat in log scale): %s\n",
  signif(log_summary$coefficients[1,1], 4)))
cat(sprintf("Slope (beta1_hat in log scale):      %s\n",
  signif(log_summary$coefficients[2,1], 4)))
cat(sprintf("Log-Model Fit Parameter (R^2):      %s\n",
  signif(log_summary$r.squared, 4)))

# resid(lm.Watershed)

opar <- par(mfrow = c(2, 2), oma = c(0, .3, 1.3, 0))

plot(lm.Watershed); par(opar)

qqp(lm.Watershed, reps = 10000)

residualPlots(lm.Watershed, smooth = T, id = T)

ols_test_breusch_pagan(lm.Watershed)

ols_test_f(lm.Watershed)

ols_test_normality(lm.Watershed)

ols_test_outlier(lm.Watershed)

outlierTest(lm.Watershed)

ols_test_score(lm.Watershed)

influence.measures(lm.Watershed)

summary(influence.measures(lm.Watershed))

case(lm.Watershed)

```

```
cooks.distance(lm.Watershed)
```

```
covratio(lm.Watershed)
```

```
dfbetas(lm.Watershed)
```

```
dffits(lm.Watershed)
```

```
hatvalues(lm.Watershed)
```

```
ols_plot_cooksd_chart(lm.Watershed)
```

```
ols_plot_dfbetas(lm.Watershed)
```

```
ols_plot_dffits(lm.Watershed)
```

```
ols_plot_diagnostics(lm.Watershed)
```

```
ols_plot_hadi(lm.Watershed)
```

```
ols_plot_resid_box(lm.Watershed)
```

```
ols_plot_resid_fit(lm.Watershed)
```

```
ols_plot_resid_fit_spread(lm.Watershed)
```

```
ols_plot_resid_hist(lm.Watershed)
```

```
ols_plot_resid_lev(lm.Watershed)
```

```
ols_plot_resid_pot(lm.Watershed)
```

```
ols_plot_resid_qq(lm.Watershed)
```

```
ols_plot_resid_regressor(lm.Watershed)
```

```
ols_plot_resid_spread(lm.Watershed)
```

```

ols_plot_resid_stand(lm.Watershed)

ols_plot_resid_stud(lm.Watershed)

ols_plot_resid_stud_fit(lm.Watershed)

salt_concentration0 <- salt_concentration + c(rep(0, 11), 8, rep(0, 8))

(Watershed0 <- data.frame(salt_concentration0, roadway_area))

cor(salt_concentration0, roadway_area)

spm(Watershed0)

sp(roadway_area, salt_concentration0, "xy", T, T, T, T,
  xlab = "Roadway area (%)", ylab = "Salt conc. (mg/l)")

regresidplot(roadway_area, salt_concentration0, "square",
  xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")

ols_regress(salt_concentration0 ~ roadway_area, Watershed0)

(lm.Watershed0 <- lm(salt_concentration0 ~ roadway_area, Watershed0))

opar <- par(mfrow = c(2, 2), oma = c(0, .3, 1.3, 0))

plot(lm.Watershed0); par(opar)

summary(lm.Watershed0)

anova(lm.Watershed0)

lht(lm.Watershed0, "roadway_area")

confint(lm.Watershed0)

```

```
ci.plot(lm.Watershed0, main = "95% confidence and prediction intervals",
        xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")

residualPlots(lm.Watershed0, smooth = T, id = T)

ols_test_breusch_pagan(lm.Watershed0)

ols_test_f(lm.Watershed0)

ols_test_normality(lm.Watershed0)

ols_test_outlier(lm.Watershed0)

outlierTest(lm.Watershed0)

ols_test_score(lm.Watershed0)

qqp(lm.Watershed0, reps = 10000)

influence.measures(lm.Watershed0)

summary(influence.measures(lm.Watershed0))

case(lm.Watershed0)

cooks.distance(lm.Watershed0)

covratio(lm.Watershed0)

dfbetas(lm.Watershed0)

dffits(lm.Watershed0)

hatvalues(lm.Watershed0)

ols_plot_cooksd_chart(lm.Watershed0)
```



```
ols_plot_dfbetas(lm.Watershed0)
```

```
ols_plot_dffits(lm.Watershed0)
```

```
ols_plot_diagnostics(lm.Watershed0)
```

```
ols_plot_hadi(lm.Watershed0)
```

```
ols_plot_resid_box(lm.Watershed0)
```

```
ols_plot_resid_fit(lm.Watershed0)
```

```
ols_plot_resid_fit_spread(lm.Watershed0)
```

```
ols_plot_resid_hist(lm.Watershed0)
```

```
ols_plot_resid_lev(lm.Watershed0)
```

```
ols_plot_resid_pot(lm.Watershed0)
```

```
ols_plot_resid_qq(lm.Watershed0)
```

```
ols_plot_resid_regressor(lm.Watershed0)
```

```
ols_plot_resid_spread(lm.Watershed0)
```

```
ols_plot_resid_stand(lm.Watershed0)
```

```
ols_plot_resid_stud(lm.Watershed0)
```

```
ols_plot_resid_stud_fit(lm.Watershed0)
```

```
# To analyze this data set, one possible way is first to remove (x12, y12)  
# and then to analyze the remaining data set in Table 6-1.
```

```
# Figure 6-16 Effect of an influential observation.
```

```

salt_concentrationI <- salt_concentration + c(rep(0, 18), 30, 0)

(WatershedI <- data.frame(salt_concentrationI, roadway_area))

cor(salt_concentrationI, roadway_area)

spm(WatershedI)

sp(roadway_area, salt_concentrationI, "xy", T, T, T, T,
  xlab = "Roadway area (%)", ylab = "Salt conc. (mg/l)")

regresidplot(roadway_area, salt_concentrationI, "square",
  xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")

ols_regress(salt_concentrationI ~ roadway_area, WatershedI)

(lm.WatershedI <- lm(salt_concentrationI ~ roadway_area, WatershedI))

opar <- par(mfrow = c(2, 2), oma = c(0, .3, 1.3, 0))

plot(lm.WatershedI); par(opar)

summary(lm.WatershedI)

anova(lm.WatershedI)

lht(lm.WatershedI, "roadway_area")

confint(lm.WatershedI)

ci.plot(lm.WatershedI, main = "95% confidence and prediction intervals",
  xlab = "Roadway area (%)", ylab = "Salt Conc. (mg/l)")

residualPlots(lm.WatershedI, smooth = T, id = T)

(lm.Watershed2I <- lm(salt_concentrationI ~ roadway_area +
  I(roadway_area^2), WatershedI))

```

```
opar <- par(mfrow = c(2, 2), oma = c(0, .3, 1.3, 0))

plot(lm.Watershed2I); par(opar)

summary(lm.Watershed2I)

anova(lm.Watershed2I)

lht(lm.Watershed2I, "I(roadway_area^2)")

lht(lm.Watershed2I, c("roadway_area", "I(roadway_area^2)"))

ols_test_breusch_pagan(lm.WatershedI)

ols_test_f(lm.WatershedI)

ols_test_normality(lm.WatershedI)

ols_test_outlier(lm.WatershedI)

outlierTest(lm.WatershedI)

ols_test_score(lm.WatershedI)

qqp(lm.WatershedI, reps = 10000)

influence.measures(lm.WatershedI)

summary(influence.measures(lm.WatershedI))

case(lm.WatershedI)

cooks.distance(lm.WatershedI)

covratio(lm.WatershedI)

dfbetas(lm.WatershedI)
```

```
dffits(lm.WatershedI)
```

```
hatvalues(lm.WatershedI)
```

```
ols_plot_cooksd_chart(lm.WatershedI)
```

```
ols_plot_dfbetas(lm.WatershedI)
```

```
ols_plot_dffits(lm.WatershedI)
```

```
ols_plot_diagnostics(lm.WatershedI)
```

```
ols_plot_hadi(lm.WatershedI)
```

```
ols_plot_resid_box(lm.WatershedI)
```

```
ols_plot_resid_fit(lm.WatershedI)
```

```
ols_plot_resid_fit_spread(lm.WatershedI)
```

```
ols_plot_resid_hist(lm.WatershedI)
```

```
ols_plot_resid_lev(lm.WatershedI)
```

```
ols_plot_resid_pot(lm.WatershedI)
```

```
ols_plot_resid_qq(lm.WatershedI)
```

```
ols_plot_resid_regressor(lm.WatershedI)
```

```
ols_plot_resid_spread(lm.WatershedI)
```

```
ols_plot_resid_stand(lm.WatershedI)
```

```
ols_plot_resid_stud(lm.WatershedI)
```

```
ols_plot_resid_stud_fit(lm.WatershedI)
```

To analyze this data set, one possible way is first to remove (x19, y19)
and then to analyze the remaining data set in Table 6-1.

```
# § 6-2.6 Correlation and Regression
```

```
# Refer to § 6-2.1 and 6-2.5.
```

```
# EXAMPLE 6-6 (continued)
```

```
# 1. Run the correlation analysis
```

```
cor_results <- cor.test(roadway_area, salt_concentration,  
                        alternative = "two.sided",  
                        method = "pearson",  
                        conf.level = 0.95)
```

```
# 2. Extract and print outputs with 4 significant figures
```

```
cat("--- cor.test() Outputs ---\n")  
cat(sprintf("Sample Correlation Coefficient (r): %s\n",  
            signif(cor_results$estimate, 4)))  
cat(sprintf("Calculated Test Statistic (t0): %s\n",  
            signif(cor_results$statistic, 4)))  
cat(sprintf("Degrees of Freedom (df): %d\n", cor_results$parameter))  
cat(sprintf("Exact P-Value: %.3e\n", cor_results$p.value))
```

```
cat(sprintf("Approximate 95%% CI Lower Limit: %s\n",  
            signif(cor_results$conf.int[1], 4)))  
cat(sprintf("Approximate 95%% CI Upper Limit: %s\n",  
            signif(cor_results$conf.int[2], 4)))
```

```
# 3. Derive the exact two-tailed critical value for comparison
```

```
alpha <- 0.05  
df <- cor_results$parameter  
t_critical <- qt(1 - alpha/2, df)  
cat(sprintf("Two-Tailed Critical Value ( $t_{\alpha/2}$ ): %s\n",  
            signif(t_critical, 4)))
```

```
# 4. Fit the ordinary least-squares simple linear regression model
```

```
regression_model <- lm(salt_concentration ~ roadway_area)  
model_summary <- summary(regression_model)
```

```
# 5. Extract regression slope coefficients and statistics
slope_estimate <- model_summary$coefficients["roadway_area", "Estimate"]
slope_se       <- model_summary$coefficients["roadway_area", "Std. Error"]
slope_t0       <- model_summary$coefficients["roadway_area", "t value"]
slope_pvalue   <- model_summary$coefficients["roadway_area", "Pr(>|t|)"]
residual_df    <- model_summary$df[2]
```

```
# 6. Print out the structured comparison table
```

```
cat("=====\n")
cat(" STATISTICAL METRIC COMPARISON SUMMARY \n")
cat("=====\n")
cat(sprintf("Degrees of Freedom (DF): %d\n", residual_df))
cat(sprintf("Slope Estimate (beta_hat): %s\n",
  signif(slope_estimate, 4)))
cat(sprintf("Slope Estimated Standard Error (SE): %s\n",
  signif(slope_se, 4)))
cat("\n--- Test Statistic (t0) Verification ---\n")
cat(sprintf("Regression Slope T-Test Statistic: %s\n",
  signif(slope_t0, 4)))
cat(sprintf("Correlation Coefficient Test Statistic: %s\n",
  signif(cor_results$statistic, 4)))
cat("\n--- P-Value Verification ---\n")
cat(sprintf("Regression Slope T-Test P-Value: %.3e\n", slope_pvalue))
cat(sprintf("Correlation Coefficient Test P-Value: %.3e\n",
  cor_results$p.value))
cat("=====\n")
```

```
spm(Watershed, T, T, T, T)
```

```
sp(roadway_area, salt_concentration, "xy", T, T, T, T,
  xlab = "Roadway area (%)", ylab = "Salt conc. (mg/l)")
```

```
sp(salt_concentration, roadway_area, "xy", T, T, T, T,
  xlab = "Salt conc. (mg/l)", ylab = "Roadway area (%)")
```

```
bivariateNormal(cor(roadway_area, salt_concentration))
```

```
ad.test(roadway_area); cvm.test(roadway_area)
```

```
lillie.test(roadway_area); pearson.test(roadway_area)
```

```
sf.test(roadway_area); shapiro.test(roadway_area)
```

```
qqp(roadway_area)
```

```
ad.test(salt_concentration); cvm.test(salt_concentration)
```

```
lillie.test(salt_concentration); pearson.test(salt_concentration)
```

```
sf.test(salt_concentration); shapiro.test(salt_concentration)
```

```
qqp(salt_concentration)
```

```
qqp(lm(salt_concentration ~ roadway_area), reps = 10000)
```

```
qqp(lm(roadway_area ~ salt_concentration), reps = 10000)
```

```
# 7. Input the baseline data
```

```
r <- 0.9754034
```

```
n <- 20
```

```
alpha <- 0.05
```

```
# 8. Step 1: Apply Fisher's Z-transformation to the sample r
```

```
# This can be calculated via  $\log((1+r)/(1-r)) * 0.5$  or R's built-in atanh()
```

```
# function
```

```
Z_r <- atanh(r)
```

```
# 9. Step 2: Calculate the standard error in the Z-scale
```

```
se_Z <- 1 / sqrt(n - 3)
```

```
# 10. Step 3: Construct the 95% confidence limits in the Z-scale
```

```
z_critical <- qnorm(1 - alpha/2)
```

```
Z_L <- Z_r - z_critical * se_Z
```

```
Z_U <- Z_r + z_critical * se_Z
```



```
# 11. Step 4: Back-transform the limits back to the rho scale
# This can be done via (exp(2*Z) - 1) / (exp(2*Z) + 1) or R's built-in
# tanh() function
rho_L <- tanh(Z_L)
rho_U <- tanh(Z_U)

# Print high-precision numerical results
cat(sprintf("Fisher's Z_r: %.3f\n", Z_r))
cat(sprintf("95%% Confidence Interval for rho: [%.4f, %.4f]\n", rho_L,
rho_U))

# EXERCISES FOR § 6-2
```

```
# § 6-3 Multiple Regression
```

```
# Refer to § 1-3, 6-1 and 6-2.
```

```
# --- 1. Setup ---
```

```
if(!require(pacman)) install.packages("pacman")
pacman::p_load(broom, car, ggfortify, ggplot2, goftest, HH, lmtest, mgcv,
  nortest, olsrr, patchwork, rgl)
```

```
# --- 2. Data Preparation ---
```

```
# Wire bond data
```

```
(WB_df <- data.frame(
  PullStr = c( 9.95, 24.45, 31.75, 35.00, 25.02, 16.86, 14.38,  9.60,
              24.35, 27.50, 17.08, 37.00, 41.95, 11.66, 21.65, 17.89,
              69.00, 10.30, 34.93, 46.59, 44.88, 54.12, 56.63, 22.13,
              21.15),
  WireLn  = c( 2,  8, 11, 10,  8,  4,  2,  2,  9,  8,  4, 11, 12,  2,  4,
              4, 20,  1, 10, 15, 15, 16, 17,  6,  5),
  DieHt   = c( 50, 110, 120, 550, 295, 200, 375,  52, 100, 300, 412, 400,
              500, 360, 205, 400, 600, 585, 540, 250, 290, 510, 590, 100,
              400)
))
```

```
# --- 3. Exploratory Analysis (Fig 1-11 / 6-17) ---
```

```
scatterplotMatrix(~ PullStr + WireLn + DieHt, data = WB_df,
  smooth = list(lty.smooth = 1, col.smooth = "red"),
  regLine = list(col = "blue"),
  main = "Figure 1-11: Wire Bond Scatterplot Matrix")
```

```
# --- 4. 3D Visualization (Fig 1-12) ---
```

```
# Note: PullStr is the dependent variable (Y-axis in car::scatter3d)
```

```
scatter3d(PullStr ~ WireLn + DieHt, data = WB_df,
  surface = FALSE, grid = TRUE, sphere.size = 1,
  xlab = "Wire Length", ylab = "Pull Strength",
  zlab = "Die Height")
```

```

# --- 5. Modeling & Selection ---
lm_lin <- lm(PullStr ~ WireLn + DieHt, data = WB_df)
summary(lm_lin)

# Check for non-linearity
residualPlots(lm_lin, smooth = TRUE, id = TRUE)

# Automated Stepwise Selection (AIC based)
# Using ols_step_both_aic requires a model object
lm_quad_full <- lm(PullStr ~ (WireLn + DieHt)^2 + I(WireLn^2) + I(DieHt^2),
                  data = WB_df)
summary(lm_quad_full)
(step_aic_result <- ols_step_both_aic(lm_quad_full))
plot(step_aic_result)

# Final model (Empirical model usually focuses on main effects +
# interaction)
lm_final <- lm(PullStr ~ WireLn + DieHt + I(WireLn*DieHt), data = WB_df)
summary(lm_final)

# --- 6. Diagnostics ---
vif(lm_lin) # Check multicollinearity on linear terms
par(mfrow = c(2, 2), oma = c(0, 0.3, 1.5, 0))
plot(lm_final)
par(mfrow = c(1, 1))

qqPlot(lm_final, main = "Residual Normality", reps = 10000)

# --- 7. 3D Surface Visualizations (Figure 1-13) ---
# This visualizes the regression plane/surface
scatter3d(PullStr ~ WireLn + DieHt, data = WB_df,
          fit = c("linear", "quadratic"),
          surface.col = c("steelblue", "orange"), alpha = 0.5,
          xlab = "Wire Length", ylab = "Pull Strength",
          zlab = "Die Height", id = TRUE)

```

§ 6-3.1 Estimation of Parameters in Multiple Regression

Refer to § 6-2.1.

Table 6-4 Data for Multiple Linear Regression

EXAMPLE 6-7 Wire Bond Pull Strength

Refer to Table 1-1 Wire Bond Data & Figures 1-11~13 in § 1-3.

Table 1-1 Wire Bond Data

Figure 1-11 Scatter plots of the wire bond pull strength data in Table
1-1.

Figure 1-12 Three-dimensional scatter diagram of the wire and pull
strength data.

Figure 1-13 Plot of estimated values of pull strength from the empirical
model in equation 1-6.

Refer to Table 2-9 Wire Bond Data & Figure 2-19 in § 2-6.

Table 2-9 Wire Bond Data

Figure 2-19 Scatter diagrams and box plots for the wire bond pull
strength data in Table 1-1. (a) Pull strength versus length. (b) Pull
strength versus die height.

Table 6-5 Wire Bond Pull Strength Data for Example 6-7

```
PullStr <- WB_df$PullStr
```

```
WireLn <- WB_df$WireLn
```

```
DieHt <- WB_df$DieHt
```

```

summary(WB_df)

# Figure 6-17 Matrix of scatter plots (from Minitab) for the wire bond pull
# strength data in Table 6-5.

sp(WireLn, PullStr, "xy", xlab = "Wire length", ylab = "Pull strength")

sp(DieHt, PullStr, "xy", xlab = "Die height", ylab = "Pull strength")

scatter3d(WireLn, PullStr, DieHt, "Wire length", "Pull strength",
"Die height", T, T, surface = F)

scatter3d(WireLn, PullStr, DieHt, "Wire length", "Pull strength",
"Die height", T, T, model.summary = T)

scatter3d(WireLn, PullStr, DieHt, "Wire length", "Pull strength",
"Die height", T, T, fit = "q", model.summary = T)

ols_regress(PullStr ~ WireLn + DieHt + I(WireLn*DieHt), WB_df)

(lm.WB <- lm(PullStr ~ WireLn + DieHt + I(WireLn*DieHt), WB_df))

residualPlots(lm.WB, smooth = T, id = T)

qqp(lm.WB, reps = 10000)

opar <- par(mfrow = c(2, 2), oma = c(0, 0, 1.1, 0)); plot(lm.WB); par(opar)

summary(lm.WB)

anova(lm.WB)

lht(lm.WB, "I(WireLn * DieHt)")

lht(lm.WB, c("WireLn", "DieHt", "I(WireLn * DieHt)"))

fitted(lm.WB)

```

```
predict(lm.WB, WB_df, T)
```

```
resid(lm.WB)
```

```
confint(lm.WB)
```

```
(newdata <- data.frame(WireLn = c(11, 5), DieHt = c(350, 200)))
```

```
predict(lm.WB, newdata, T, NULL, Inf, "c")
```

```
predict(lm.WB, newdata, T, NULL, Inf, "p")
```

```
# EXAMPLE 6-7 (continued) Compute Estimate of Error Variance  $\hat{\sigma}^2$ 
```

```
# Table 6-6 Minitab Regression Analysis Output for Wire Bond Pull Strength
```

```
# Data
```

```
# EXAMPLE 6-7 (continued)
```

§ 6-3.2 Inferences in Multiple Regression

Refer to § 6-3.1.

EXAMPLE 6-7 (continued)

Table 6-7 Minitab Output

EXAMPLE 6-7 (continued)

```
# § 6-3.3 Checking Model Adequacy
```

```
# Refer to § 6-2.5 and 6-3.1.
```

```
ols_test_breusch_pagan(lm.WB)
```

```
ols_test_f(lm.WB)
```

```
ols_test_normality(lm.WB)
```

```
ols_test_outlier(lm.WB)
```

```
outlierTest(lm.WB)
```

```
ols_test_score(lm.WB)
```

```
influence.measures(lm.WB)
```

```
summary(influence.measures(lm.WB))
```

```
case(lm.WB)
```

```
cooks.distance(lm.WB)
```

```
covratio(lm.WB)
```

```
dfbetas(lm.WB)
```

```
dffits(lm.WB)
```

```
hatvalues(lm.WB)
```

```
ols_coll_diag(lm.WB)
```

```
ols_plot_added_variable(lm.WB)
```

```
ols_plot_cooksd_chart(lm.WB)
```



```
ols_plot_dfbetas(lm.WB)

ols_plot_dffits(lm.WB)

ols_plot_diagnostics(lm.WB)

ols_plot_hadi(lm.WB)

ols_plot_resid_box(lm.WB)

ols_plot_resid_fit(lm.WB)

ols_plot_resid_fit_spread(lm.WB)

ols_plot_resid_hist(lm.WB)

ols_plot_resid_lev(lm.WB)

ols_plot_resid_pot(lm.WB)

ols_plot_resid_qq(lm.WB)

ols_plot_resid_regressor(lm.WB)

ols_plot_resid_spread(lm.WB)

ols_plot_resid_stand(lm.WB)

ols_plot_resid_stud(lm.WB)

ols_plot_resid_stud_fit(lm.WB)

# EXAMPLE 6-7 (continued)

# Figure 6-18 Normal probability plot of residuals for wire bond empirical
# model.
```

Figure 6-19 Plot of residuals against for wire bond empirical model.

Figure 6-20 Plot of residuals against x1 (wire length) for wire bond
empirical model.

Figure 6-21 Plot of residuals against x2 (die height) for wire bond
empirical model.

EXAMPLE 6-7 (continued)

Figure 6-22 A point that is remote in x-space.

EXAMPLE 6-8 Wire Bond Pull Strength

Table 6-8 Influence Diagnostics for the Wire Bond Pull Strength Data

EXAMPLE 6-8 (continued)

EXERCISES FOR § 6-3

```
# § 6-4 OTHER ASPECTS OF REGRESSION
```

```
# Refer to § 6-1, 6-2, and 6-3.
```

```
# --- 1. Setup ---
```

```
if(!require(pacman)) install.packages("pacman")
```

```
pacman::p_load(broom, car, ggfortify, ggplot2, goftest, HH, lmtest, mgcv,  
  nortest, olsrr, patchwork, rgl)
```

```
# § 6-4.1 Polynomial Models
```

```
# EXAMPLE 6-9 Acetylene Yield
```

```
# Table 6-9 The Acetylene Data
```

```
Yld <- c(49.0, 50.2, 50.5, 48.5, 47.5, 44.5, 28.0, 31.5, 34.5, 35.0, 38.0,  
38.5, 15.0, 17.0, 20.5, 29.5)
```

```
Temp <- c(1300, 1300, 1300, 1300, 1300, 1300, 1200, 1200, 1200, 1200, 1200,  
1200, 1100, 1100, 1100, 1100)
```

```
Rto <- c(7.5, 9.0, 11.0, 13.5, 17.0, 23.0, 5.3, 7.5, 11.0, 13.5, 17.0,  
23.0, 5.3, 7.5, 11.0, 17.0)
```

```
(C2H2 <- data.frame(Yld, Temp, Rto))
```

```
summary(C2H2); var(C2H2); cor(C2H2)
```

```
cor(Yld, Temp); cor(Yld, Rto); cor(Temp, Rto)
```

```
spm(C2H2)
```

```
sp(Temp, Yld, "xy", xlab = "Temperature", ylab = " Yield")
```

```
sp(Rto, Yld, "xy", xlab = "Ratio", ylab = " Yield")
```

```
scatter3d(Temp, Yld, Rto, "Temperature", " Yield", "Ratio", T, T,  
surface = F)
```

```
scatter3d(Temp, Yld, Rto, "Temperature", " Yield", "Ratio", T, T,  
model.summary = T)
```

```
scatter3d(Temp, Yld, Rto, "Temperature", " Yield", "Ratio", T, T,  
fit = "q", model.summary = T)
```

```
TempC <- Temp - mean(Temp); RtoC <- Rto - mean(Rto)
```

```

C2H2C <- data.frame(Yld, TempC, RtoC)

ols_regress(Yld ~ TempC + RtoC + I(TempC*RtoC), C2H2C)

(lm.C2H2C <- lm(Yld ~ TempC + RtoC + I(TempC*RtoC), C2H2C))

residualPlots(lm.C2H2C, smooth = T, id = T)

ols_regress(Yld ~ TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC*RtoC),
C2H2C)

lm.C2H2C2 <- lm(Yld ~
TempC + RtoC + I(TempC^2) + I(RtoC^2) + I(TempC*RtoC), C2H2C)

opar <- par(mfrow = c(2, 2), oma = c(0, 0, 1.1, 0))

plot(lm.C2H2C2); par(opar)

summary(lm.C2H2C2)

ols_coll_diag(lm.C2H2C2)

anova(lm.C2H2C2)

lht(lm.C2H2C2, "I(TempC^2)")

lht(lm.C2H2C2, "I(RtoC^2)")

lht(lm.C2H2C2, "I(TempC * RtoC)")

lht(lm.C2H2C2, c("I(TempC^2)", "I(RtoC^2)"))

lht(lm.C2H2C2, c("I(TempC^2)", "I(TempC * RtoC)"))

lht(lm.C2H2C2, c("I(RtoC^2)", "I(TempC * RtoC)"))

lht(lm.C2H2C2, c("I(TempC^2)", "I(RtoC^2)", "I(TempC * RtoC)"))

```

```
lht(lm.C2H2C2,  
c("TempC", "RtoC", "I(TempC^2)", "I(RtoC^2)", "I(TempC * RtoC)"))
```

```
fitted(lm.C2H2C2)
```

```
predict(lm.C2H2C2, C2H2C, T)
```

```
resid(lm.C2H2C2)
```

```
confint(lm.C2H2C2)
```

```
# EXAMPLE 6-9 (continued)
```

§ 6-4.2 Categorical Regressors

EXAMPLE 6-10 Gas Mileage

Use Indicator Variables to Model Interaction

EXAMPLE 6-11 Shampoo Data

Refer to § 2-6.

§ 6-4.3 Variable Selection Techniques

EXAMPLE 6-12 All Possible Regressions

Table 6-10 Minitab Best Subsets Regression for Shampoo Data

EXAMPLE 6-13 Stepwise Regression

Table 6-11 Stepwise Regression Backward Elimination for Shampoo Data:

Quality versus Foam, Scent, Color, Residue, Region

Table 6-12 Stepwise Regression Forward Selection for Shampoo Data:

Quality versus Foam, Scent, Color, Residue, Region

Table 6-13 Stepwise Regression Combined Forward and Backward Elimination:

Quality versus Foam, Scent, Color, Residue, Region

EXERCISES FOR § 6-4